

STRUCTURA SI ORGANIZAREA CALCULATOARELOR

Calculatoare 3 2025-2026

CURS 13 Sapt 14 20 ianuarie 2026 14:00-16:00

serban@upit.ro

Modul de notare:

Examen:	50%	
Proiect	20%	A3
Laborator:	20%	A2
Test (Midterm):	10%	A1

Obs. Fiecare componenta a notei finale (A1, A2, A3 si Examen) trebuie sa fie promovata cu nota minima 5.

Planificarea activitatilor:

Laborator:

- saptamanal, conform orarului;
- sapt. 13, 14 refaceri laboratoare;

Proiect:

- conform orarului

Curs:

- saptamanal, conform orarului;

Test (Midterm): in saptamana a 10-a (9 decembrie 2025)

Continut

Bazele aritmetice ale calculatoarelor electronice

- Sisteme de numeratie

- Reprezentarea numerelor in virgula fixa / mobila

- Operatii aritmetice in virgula fixa / mobila

Proiectarea blocurilor logice pentru operatiile aritmetice in calculatoarele electronice

- Adunarea; scaderea

- Inmultirea

- Impartirea

Proiectarea procesoarelor

- Procesoare RISC

- Setul de instructiuni

- Calea de date (Data Path)

- Calea de control (Control Path)

Bibliografie

David A. Patterson, John L. Hennessy - Computer Organization and Design The Hardware Software Interface [RISC-V Edition] (6th edition Morgan Kaufmann, 2018)

David Harris, Sarah Harris - Digital Design and Computer Architecture (2007)

J. Hennessy, D. Patterson - Computer Architecture. A Quantitative Approach (6th edition Morgan Kaufmann 2019)

William Stallings - Computer Organization and Architecture Designing for Performance (10th edition Pearson 2016)

A. Tanenbaum, T. Austin - Structured Computer Organization, (6th edition Pearson 2013)

Karbo M.B. - A complete illustrated guide to the PC hardware (2005)

Proiectarea unui MPU de tip RISC

RISC – Reduced Instruction Set Computer

CISC - Complex Instruction Set Computer

Un program este o secvență de instrucțiuni care permite procesarea de informații de către un procesor.

Efectuarea unei instrucțiuni se poate realiza în 2 moduri:

- prin interpretare, iar în acest sens instrucțiunea se descompune în microinstrucțiuni;
- prin executare, instrucțiunea fiind realizată direct de structura hardware (cablat), fără vreo intermediere.

Procesor interpretor – operează folosind logica microprogramată, fiecare instrucțiune fiind executată într-un automat microprogramat prin intermediul mai multor microinstrucțiuni ce alcătuiesc microprogram.

Procesor executiv – operează folosind logica cablată, fiecare instrucțiune fiind executată direct de hardware, fără folosirea de automate microprogramate.

Un MPU RISC este un procesor executiv care execută direct instrucțiunile primite, fără a le interpreta folosindu-se de structura sa internă și fără a folosi automate microprogramate.

Proiectarea unui MPU RISC presupune parcurgerea următoarelor activități:

- Proiectarea setului de instrucțiuni (instruction set architecture);
- Proiectarea căii de date (data path), adică a structurilor digitale și respectiv a magistrelor de comunicații dintre acestea care permit executia propriu-zisă a instrucțiunilor;
- Proiectarea căii de control (control path), adică a structurii logice care gestionează circuitele digitale ce formează calea de date, oferind comanda acestor circuite în vederea execuției instrucțiunilor.

Arhitectura unui MPU RISC este dependentă de setul de instrucțiuni pe care îl execută. O modificare de instrucțiune poate atrage schimbări majore la nivelul structurii hardware a procesorului.

Caracteristicile procesorului de tip RISC care va fi proiectat

Procesorul va avea o arhitectura Harvard cu spatii de memorie distincte pentru programe, respectiv date.

Organizarea memoriei de programe va fi de 64k x 32, iar cea a memoriei de date 64k x16.

Procesorul va dispune de 2 magistrale distincte de adrese de cate 16 biti, una fiind folosita pentru adresarea memoriei de programe, cealalta pentru adresarea memoriei de date. Va mai dispune si de 2 magistrale de date, una pe 32 de biti cu are sens de intrare in procesor, fiind folosita pentru preluarea codurilor instructiunilor (citire de catre UC), cealalta pe 16 biti, care permite tranferul de informatii intre procesor si memoria de date, aceasta fiind bidirectionala si permitand operatii de citire si scriere. Informatiile transferate pe aceasta magistrala sunt operanzi ce urmeaza a fi procesati in interiorul procesorului, in situatia citirii acestora, respectiv rezultate depuse de procesor in memoria de date pentru situatia scrierii acestora.

Procesorul va fi actionat de un semnal de tip ceas digital, fiecare instructiune urmand a fi executata intr o singura perioada a acestui semnal de ceas.

Procesorul va putea sa execute un total de 32 de instructiuni, fiecare fiind reprezentata pe un dublu cuvânt (32 biti).

Procesorul va dispune de un set de 32 de registri interni, fiecare cu dimensiunea de 16 biti, care se vor comporta similar pentru instructiunile ce formeaza setul de instructiuni (nu va exista vreun registru privilegiat). Aceasta proprietate poarta denumirea de ortogonalitatea setului de instructiuni in raport cu registrii procesorului.

Pentru simplificarea arhitecturii hardware, mecanismul stivei va fi redus la minim. In acest sens, 1 din cei 32 de registri din setul de registri, va fi utilizat pe post de stiva. Acest registru va contine varful stivei, de unde si denumirea acestuia TOS – top of stack. Registrul TOS este cel al 32-lea din setul de registri. Aceasta simplificare conduce la o stiva formata dintr-o singura locatie, fiind astfel posibila la procesorul proiectat fie apelarea unei singure subrutine, fie tratarea unei singure intreruperi, nefiind posibil apel imbricat de subrutine sau depuneri multiple in stiva.

Proiectarea unui MPU de tip RISC – cont.

1. Proiectarea setului de instructiuni (instruction set architecture)

Conform caracteristicilor enuntate, procesorul RISC propus va avea instructiunile reprezentate pe 32 de biti. Instructiunile din setul propus, vor avea urmatoarele formate binare posibile:

Formatul 1:

31	-	27	26	-	22	21	-	17	16	-	12	11	-	0
COD			d			l			r			-		

unde:

COD = codul masina al instructiunii;

d = destinatie rezultat (registru in setul de registri);

l = operand stang (registru in setul de registri);

r = operand drept (registru in setul de registri).

Formatul 1 va fi folosit pentru instructiuni de tipul:

OP d, l, r (d ← l Operatie r) depune in destinatia **d** rezultatul operatiei **OP** dintre operandii **l** si **r**.

Formatul 2:

31	-	27	26	-	22	21	-	17	16	15	-	0
COD			d			l			-	v		

unde:

COD = codul masina al instructiunii;

d = destinatie rezultat (registru in setul de registri);

l = operand stang (registru in setul de registri);

v = valoare imediata (operand imediat).

Formatul 2 va fi folosit pentru instructiuni de tipul:

OP d, l, v (d ← l Operatie v) depune in destinatia **d** rezultatul operatiei **OP** dintre operandii **l** si **v**.

Din formatele reprezentate => procesorul va efectua 2^5 instructiuni deoarece COD este format din 5 biti.

Mai rezulta ca d poate avea $2^5 = 32$ valori. Operandul l poate avea 32 de valori ca si operandul r.

Valorile care le poate lua d reprezinta adrese destinatie, de fapt vor fi registri aflati intr-un set de registri pe care procesorul il va avea in dotare. In mod similar l si r reprezinta adrese ale operandilor respectivi, de fapt registri aflati in acelasi set de registri => procesorul pe care il vom proiecta va avea un set de 32 de registri care pot fi si surse ale operandilor, dar si destinatii pentru rezultat.

Pentru ca acest lucru sa fie posibil, setul de registri va fi dotat cu un mecanism multiplu de adresare care sa permita simultan adresarea a 2 registri pe post de operand respectiv al unui registru destinatie pentru plasarea rezultatului instructiunii.

1. Proiectarea setului de instructiuni (instruction set architecture) – cont.

Instructiunile procesorului propus sunt grupate in urmatoarele categorii:

- a) Instructiuni functionale – categorie ce cuprinde instructiuni aritmetice, logice si alte instructiuni.
- b) Instructiuni de salt – cuprinde instructiuni de salt conditionat si neconditionat.
- c) Instructiuni de control

a) Instructiuni functionale:

a1). Instructiuni aritmetice:

- 1. ADD d,l,r <==> $d \leftarrow l + r$; F1
- 2. SUB d,l,r <==> $d \leftarrow l - r$; F1
- 3. ADDV d,l,v <==> $d \leftarrow l + v$; F2
- 4. SUBV d,l,v <==> $d \leftarrow l - v$; F2

a2). Instructiuni logice:

- 5. AND d,l,r <==> $d \leftarrow l \wedge r$; F1
- 6. OR d,l,r <==> $d \leftarrow l \vee r$; F1
- 7. XOR d,l,r <==> $d \leftarrow l \oplus r$; F1
- 8. NOT d,l <==> $d \leftarrow \bar{l}$; F1, F2

a3). Alte instructiuni:

- 9. SHR d,l <==> $d \leftarrow l/2$; F1, F2
- 10. SHL d,l <==> $d \leftarrow 2 * l$; F1, F2
- 11. MOV d,l <==> $d \leftarrow l$; F1, F2 (move – transfera continutul unui registru sursa intr-un registru destinatie)
- 12. LOAD d,l <==> $d \leftarrow (l)$; F1, F2 (permite citirea unei locatii de memorie externa de date a carei adresa este precizata in registrul l)
- 13. STORE l, r <==> $(l) \leftarrow r$; F1 (permite scrierea unei locatii de memorie externa de date a carei adresa este precizata in registrul l)
- 14. VAL d, v <==> $d \leftarrow v$; F2
- 15. CR d, l, r <==> $d \leftarrow Cy(l + r)$ (permite adunari cu operanzi exprimati pe mai multe cuvinte)
- 16. BR d, l, r <==> $d \leftarrow Bw(l - r)$ (permite scaderi cu operanzi exprimati pe mai multe cuvinte)

1. Proiectarea setului de instructiuni (instruction set architecture) – cont.

b) Instructiuni de salt

b1). Salturi neconditionate:

17. JMP v <==> $PC \leftarrow v$; F2 (salt absolut)
18. JMR v <==> $PC \leftarrow PC + 1 + v$; F2 (salt relativ)

Aceasta instructiune incarca PC cu o adresa a carei valoare depinde valoarea anterioara. Continutul lui v poate fi in acest caz un numar exprimat in CC2 pe 16 biti care va indica distanta de salt la care va ajunge procesorul in memoria de program, luand ca referinta adresa existenta in PC (PC +1).

b2). Lucru cu subrutine:

19. CALL v <==> $TOS \leftarrow PC + 1, PC \leftarrow v$; TOS= top of stack (subrutina cu apelare absoluta)
Apeleaza o subrutina aflata la adresa absoluta v in memoria de programe. Pentru acest lucru salveaza continutul PC-ului, adica adresa de la care se face reluarea programului dupa revenirea din subrutina, in stiva. Pentru simplificarea procesorului proiectat am considerat ca in locul stivei este folosit un unic registru din setul de registri pe care notat TOS.
20. CALR v <==> $TOS \leftarrow PC + 1, PC \leftarrow PC + 1 + v$ subrutina cu apelare relativa - Call relative); v este reprezentat in CC2
21. RET <==> $PC \leftarrow TOS$ (revenire din subrutina - Return)

b3). Salturi conditionate:

22. CJZ I, v <==> $I=0 \Rightarrow PC \leftarrow v; I \neq 0 \Rightarrow PC \leftarrow PC + 1$ F2 (compare and jump if zero)
23. CJNZ I, v <==> $I=0 \Rightarrow PC \leftarrow PC + 1; I \neq 0 \Rightarrow PC \leftarrow v$ F2 (compare and jump if not zero)
24. CJRZ I, v <==> $I=0 \Rightarrow PC \leftarrow PC + 1 + v; I \neq 0 \Rightarrow PC \leftarrow PC + 1$ F2 (compare and jump relative if zero); v este CC2
25. CJRNZ I, v <==> $I=0 \Rightarrow PC \leftarrow PC + 1; I \neq 0 \Rightarrow PC \leftarrow PC + 1 + v$ F2 (compare and jump relative if not zero) ; v este CC2

OBSERVATIE: Instructiunile cu adresare relativa permit relocarea programelor in memoria de programe a sistemului de calcul, adica mutarea acestor programe in diverse zone de memorie, pastrandu-se proprietatea de a rula in mod similar cu aceleasi rezultate, indiferent de zona de memorie in care sunt mutate. Folosind relocabilitatea se poate defragmenta memoria atunci cand este folosita pentru mai multe programe.

1. Proiectarea setului de instructiuni (instruction set architecture) – cont.

c). Instructiuni de control

- 26. HALT – oprire procesor
- 27. NOP – no operation
- 28. DI – Disable interrupt
- 29. EI – Enable interrupt

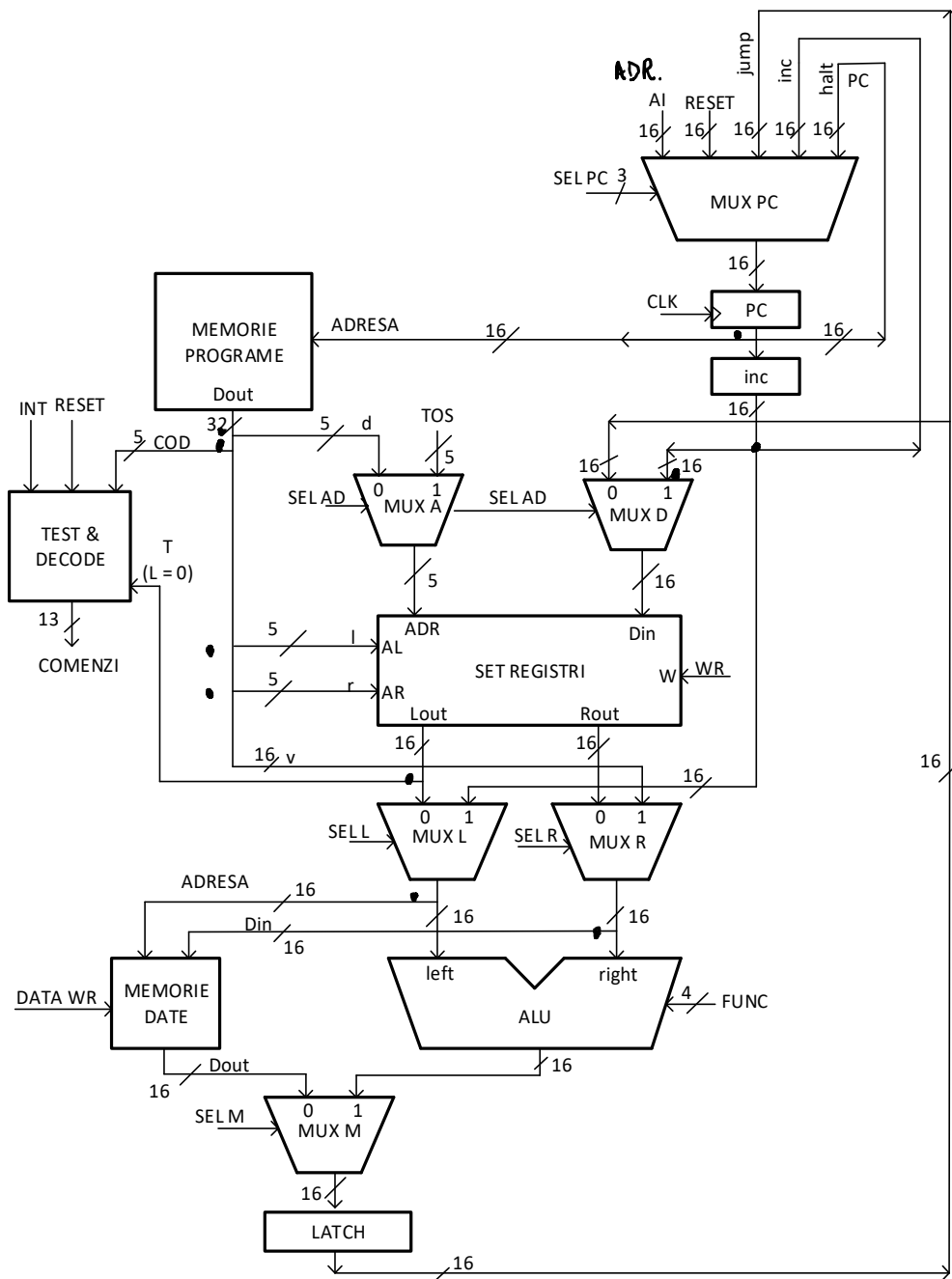
Ultimele 2 instructiuni vor permite sa implementam un mecanism de intreruperi mascabil la nivelul procesorului. Procesorul va dispune de o intrare pentru primirea cererilor hardware de intreruperi care vor putea fi validate sau invalidate pe cale software, folosind instructiunile mentionate.

La instructiunile prezentate, care se vor putea prelua din memoria de programe, se adauga inca 2 instructiuni (comenzi) declansate la nivelul procesorului ca urmare a aplicarii unor semnale externe:

- 30. RESET <==> PC = 0 (initializarea procesorului); se declanseaza ca urmare a aparitiei unui semnal de tip RESET
- 31. INT <==> TOS $\leftarrow$ PC+1 ; PC $\leftarrow$ AI (adresa subrutina tratare intreruperi) se declanseaza la primirea unei intreruperi externe (dupa rulara EI)

16 dec 2025

2. Proiectarea caii de date (data path)



Rolul componentelor

Registrul **PC – Program Counter** este utilizat pentru adresarea memoriei de programe din care se vor extrage codurile instructiunilor ce urmeaza a fi executate. Schimbarea continutului PC (trecerea la adresa instructiunii urmatoare) se realizeaza in ritmul unui semnal de ceas CLK care provine de la un oscilator digital extern (ceasul procesorului).

Schimbarea continutului PC este provocata de mai multe surse (5). Alegerea uneia dintre aceste surse se realizeaza prin utilizarea circuitului MUX PC. Selectia propriu-zisa a sursei se face prin 3 linii de comanda notate SEL PC. Aceste comenzi ca si celelalte care apar in circuit sunt generate de T&D care formeaza calea de control a structurii (Test & Decode).

O prima sursa de adresa pentru PC, este chiar continutul anterior al registrului PC. Alegand aceasta sursa continutul PC-ului ramane nemodificat pentru urmatorul semnal CLK. Situatia apare la executia instructiunii HALT.

O a doua sursa care apare frecvent este legata de incrementarea continutului PC (INC). In acest sens, atasat registrului PC, exista un bloc denumit INC care realizeaza operatia $PC \leftarrow PC + 1$. Situatia este folosita in tratarea instructiunilor care presupun parcurgerea liniara a programului, fara salturi sau intreruperi de orice natura.

O a treia sursa de adresa pentru registrul PC o prezinta o adresa de salt. Sursa este folosita la instructiuni care parcurg neliniar un program (salturi, apeluri de subrutina, reveniri de subrutina).

Sursele 4 si 5 sunt adresa de reset, adresa pentru tartare intrerupere. Selectia acestor surse este provocata de semnalele hard: RESET si INT.

2. Proiectarea caii de date (data path) – cont.

Rolul componentelor

Setul de registri se constituie într-o zonă de memorie RAM cu mai multe posibilități de acces; astfel din punct de vedere al citirii este posibil accesul simultan la 2 registre folosind 2 intrări pentru precizarea adreselor acestora. Este vorba de intrările AL și AR pentru operanții l și r , preluați din formatul codului instrucțiunii extras din memoria de programe. Ca urmare a acestei duble adresări, conținuturile așteptate apar pe liniile L_{OUT} și R_{OUT} .

O memorie cu dublu acces la citire având acest comportament este o memorie de tip biport (dual port RAM).

Setul de registre dispune și de posibilitatea scrierii unuia din registre conținute.

În acest sens circuitul dispune de semnalele:

- ADR – pentru aplicarea adresei destinație ce urmează a fi înscrisă.
- D_{IN} – pentru aplicarea datei de intrare ce urmează a fi înscrisă în registrul adresat.
- WR – comanda operației de scriere în registrul destinație adresat prin ADR și cu date aplicată la D_{IN} .

La operația de scriere, MUX A este folosit pentru specificarea adresei registrului destinație ce se va înscrie.

MUX A permite selecția între următoarele surse:

Câmpul d (D_{OUT} alege destinația din codul instrucțiunii) și TOS.

Pentru datele care se vor înscrie la destinația selectată (D_{IN}), selecția este asigurată de circuitul MUX D.

Una din aceste surse este constituită de ieșirile circuitului LATCH și care la rândul ei poate să provină de la ieșirile circuitului ALU sau memoria de date, iar cea de a doua sursă provine de la ieșirile circuitului INC ce oferă $PC+1$.

2. Proiectarea caii de date (data path) – cont.

Rolul componentelor

JMP v – structura permite trecerea operandului urmator v care se preia din codul instructiunii obtinute din memoria de program si ca urmare a traversarii circuitului MUX L caruia i se aplica o comanda de selectie corespunzatoare, SEL L. Urmeaza trecerea operandului v prin unitatea ALU ca urmare a aplicarii asupra acestuia a unei comenzi FUNC care permite trecerea neafectata a respectivului operand. Urmeaza trecerea mai departe prin circuitul MUX M si memorarea lui v in circuitul LATCH. Iesirile acestuia se aplica intrarilor de timp JUMP de la MUX PC.

JMR I – primul operand PC+1 se obtine la iesirile blocului INC si va traversa circuitul MUX R ca urmare a aplicarii unei selectii SEL R corespunzatoare, ajungand in final la o intrare ALU.

Operandul I este de fapt continutul unui registru din setul de registri care este adresat prin operandul I (coincidenta de notatie) si care se obtine ca urmare a aplicarii campului I din codul instructiunii extras din memoria de program.

Campul I se aplica intrarilor de adrese AL din setul de registri, iar operandul I cautat, obtinandu-se la iesirile L_{OUT} . Acest operand traverseaza MUX-ul L ca urmare al unei selectii corespunzatoare, notata SEL I si ajunge la randul lui la cealalta intrare ALU. La nivelul unitatii ALU se realizeaza o operatie de adunare intre operandul PC+1 si operandul I, rezultatul traversand circuitul MUX M (cu o comanda SEL M corespunzatoare) fiind memorat in circuitul LATCH si ajungand in final la intrarile de tip JUMP de la circuitul MUX PC.

2. Proiectarea caii de date (data path) – cont.

Comenzi emise de blocul TEST & DECODE (13):

- SEL PC (3)
- SEL AD (1)
- SEL L (1)
- SEL R (1)
- FUNC ALU (4)
- WR (1)
- DATA WR (1)
- SEL M (1)

3. Proiectarea caii de control (control path)

Blocul Test & Decode este de fapt o memorie ROM (logica cablata) al carei continut apare mai jos:

INTRARI
ADDRESA

IESIRI
DATA (13 biti)

	C ₄	C ₃	C ₂	C ₁	C ₀	T	INT	R	F ₃	F ₂	F ₁	F ₀	PC ₂	PC ₁	PC ₀	SEL AD	SEL L	SEL R	WR	DATA WR	SEL M	OBS:
1	0	0	0	0	0	*	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	ADD d,l,r
2	0	0	0	0	1	*	0	0	0	0	0	1	0	0	0	0	0	0	1	0	1	SUB d,l,r
3	0	0	0	1	0	*	0	0	0	0	0	0	0	0	0	0	0	1	1	0	1	ADDV d,l,v
4	0	0	0	1	1	*	0	0	0	0	0	1	0	0	0	0	0	1	1	0	1	SUBV d,l,v
5	0	0	1	0	0	*	0	0	0	0	1	0	0	0	0	0	0	0	1	0	1	AND d,l,r
6																						
7																						
8																						
9																						
10																						
11																						
12	0	1	0	1	1	*	0	0	*	*	*	*	0	0	0	0	0	*	1	0	0	LOAD d,l
13	0	1	1	0	0	*	0	0	*	*	*	*	0	0	0	*	0	0	0	1	*	STORE l,r
14																						
15																						
16																						
17	0	1	1	0	0	*	0	0	0	0	1	1	0	0	1	*	*	1	0	0	1	JMP v
18	0	1	1	0	1	*	0	0	0	0	0	0	0	0	1	*	1	1	0	0	1	JMR v
19	0	1	1	1	1	*	0	0	0	0	1	1	0	0	1	1	*	1	1	0	1	CALL v
20																						
21	1	0	0	0	0	*	0	0	0	0	1	1	0	0	1	*	*	0	0	0	1	RET
22	1	0	1	1	0	0	0	0	0	0	1	1	0	0	1	*	*	1	0	0	1	CJZ l,v (l = 0)
23	1	1	0	0	0	1	0	0	*	*	*	*	0	0	0	*	*	*	0	0	*	CJZ l,v (l != 0)
24																						
25																						
26																						
27																						
28																						
29																						
30	*	*	*	*	*	*	*	1	*	*	*	*	0	1	0	*	*	*	0	0	*	RESET
31	*	*	*	*	*	*	1	0	*	*	*	*	0	1	1	1	*	*	1	0	*	INT
32																						

INTRARI (8 intrari) – adrese pentru ROM

C4-C0 – cod instructiune

T – semnifica L=0

INT – cerere de intrerupere

R - Reset

IESIRI (13) date preluate din ROM

SEL PC (3)

SEL AD (1)

SEL L (1)

SEL R (1)

FUNC ALU (4)

WR (1)

DATA WR (1)

SEL M (1)

FUNC	F3	F2	F1	F0
ADD	0	0	0	0
SUB	0	0	0	1
AND	0	0	1	0
OUT = R	0	0	1	1
OUT = L	0	1	0	0

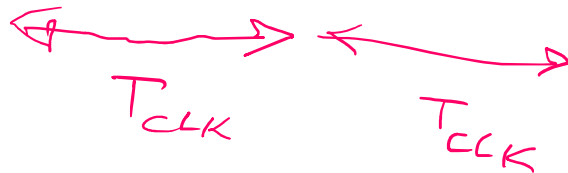
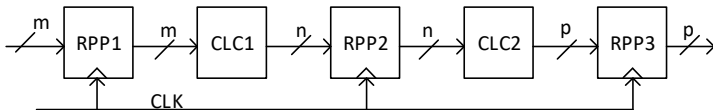
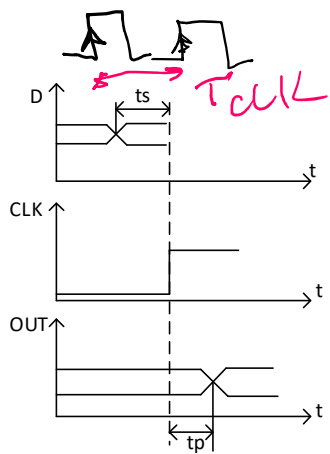
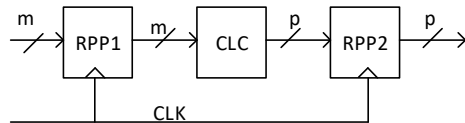
SEL PC	PC2	PC1	PC0
INC	0	0	0
JUMP	0	0	1
RESET	0	1	0
AI	0	1	1

END 13 ian 2026

SOC vineri 6 feb 2026

MLA miercuri 11 feb 2026

Mecanismul Pipeline



Calcul frecventa semnal CLK:

$$T_{CLK} = t_{pRPP1} + t_{pCLC} + t_{sRPP2};$$

$$f_{CLK} = 1/T_{CLK}$$

t_s – timp set-up

t_p – timp propagare

Pentru $t_{pRPP1} = 10\text{ns}$

$t_{pCLC} = 100\text{ns}$ $t_{sRPP2} = 5\text{ns}$

obtinem:

$$T_{CLK} = 115\text{ns};$$

$$f_{CLK} = 1/115\text{ns} = 8,69\text{MHz}$$

Calcul frecventa semnal CLK:

$$T_{CLK} = t_{pRPP} + \max(t_{pCLC1}; t_{pCLC2}) + t_{sRPP}; f_{CLK} = 1/T_{CLK}$$

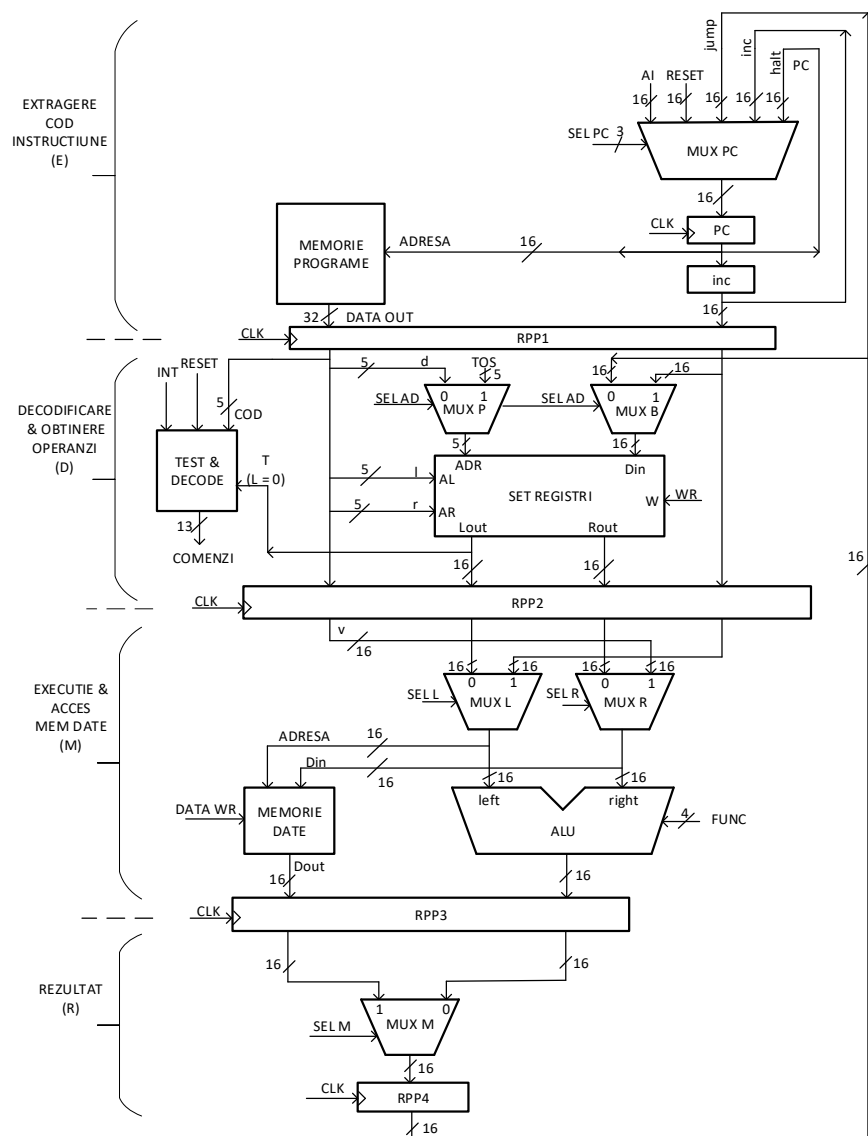
Pentru $t_{pRPP} = 10\text{ns}$ $t_{pCLC1} = 60\text{ns}$ $t_{pCLC2} = 50\text{ns}$

$\max_t_{pCLC} = 60\text{ns}$ $t_{sRPP} = 5\text{ns}$

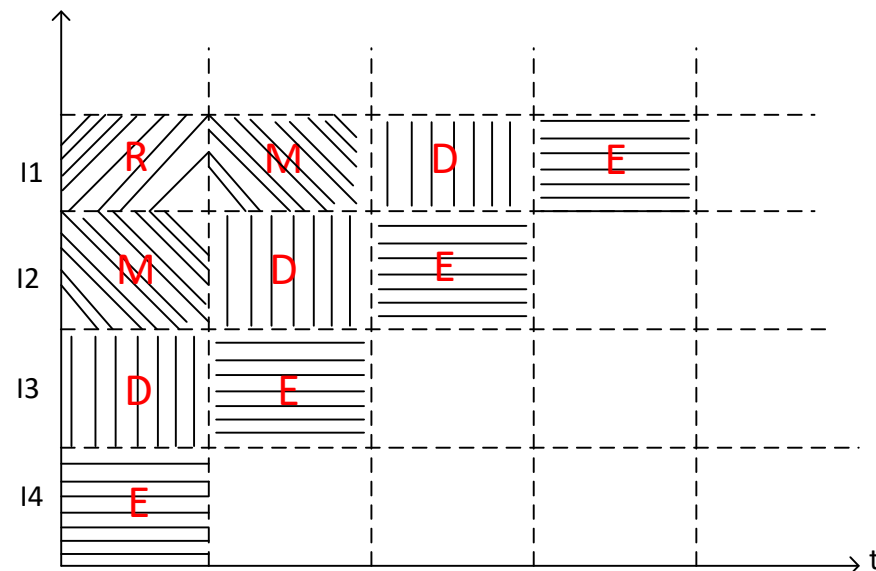
obtinem:

$$T_{CLK} = 75\text{ns}; f_{CLK} = 1/75\text{ns} = 13,33\text{MHz}$$

Se proceseaza un debit mai mare de informatii!

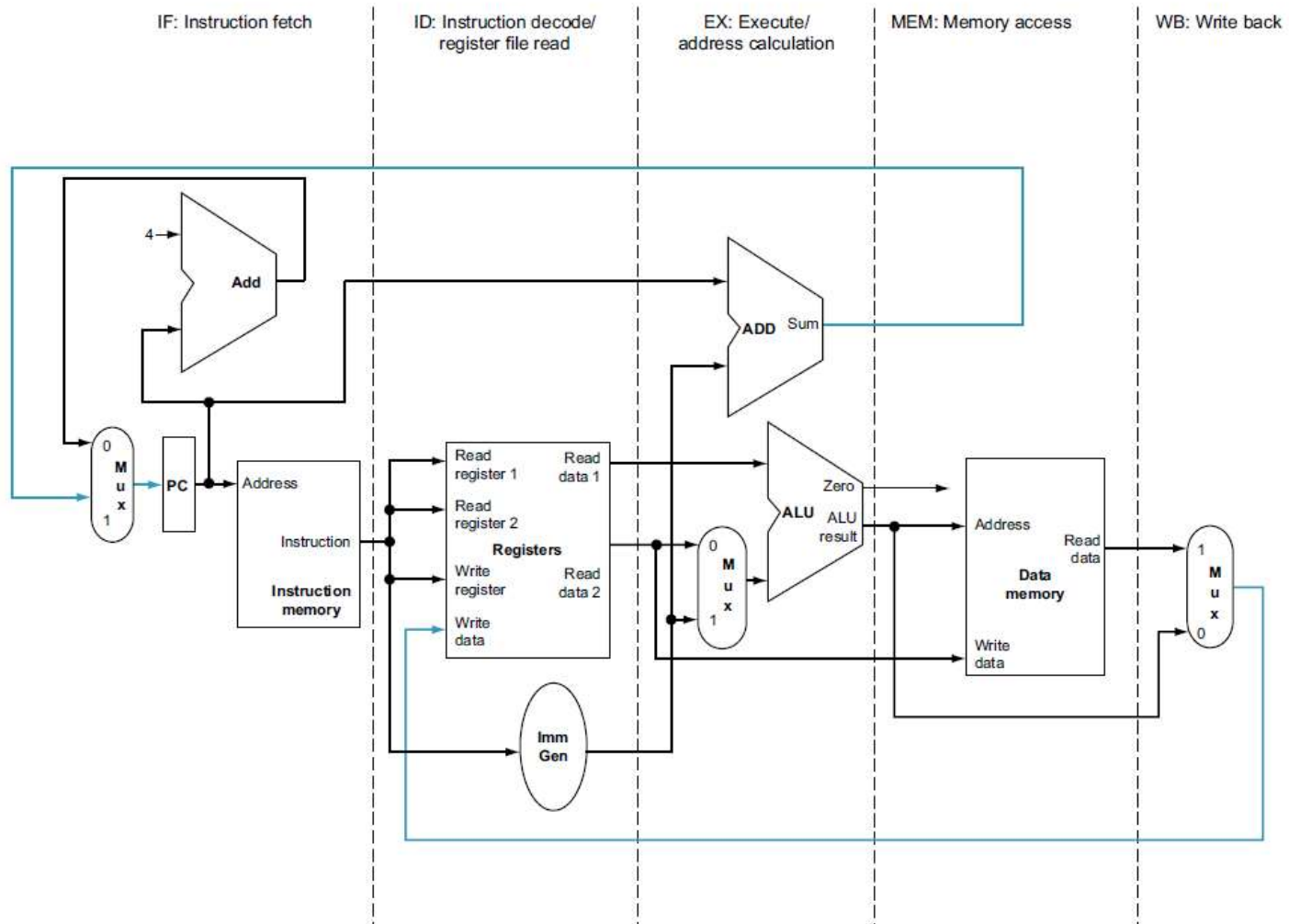


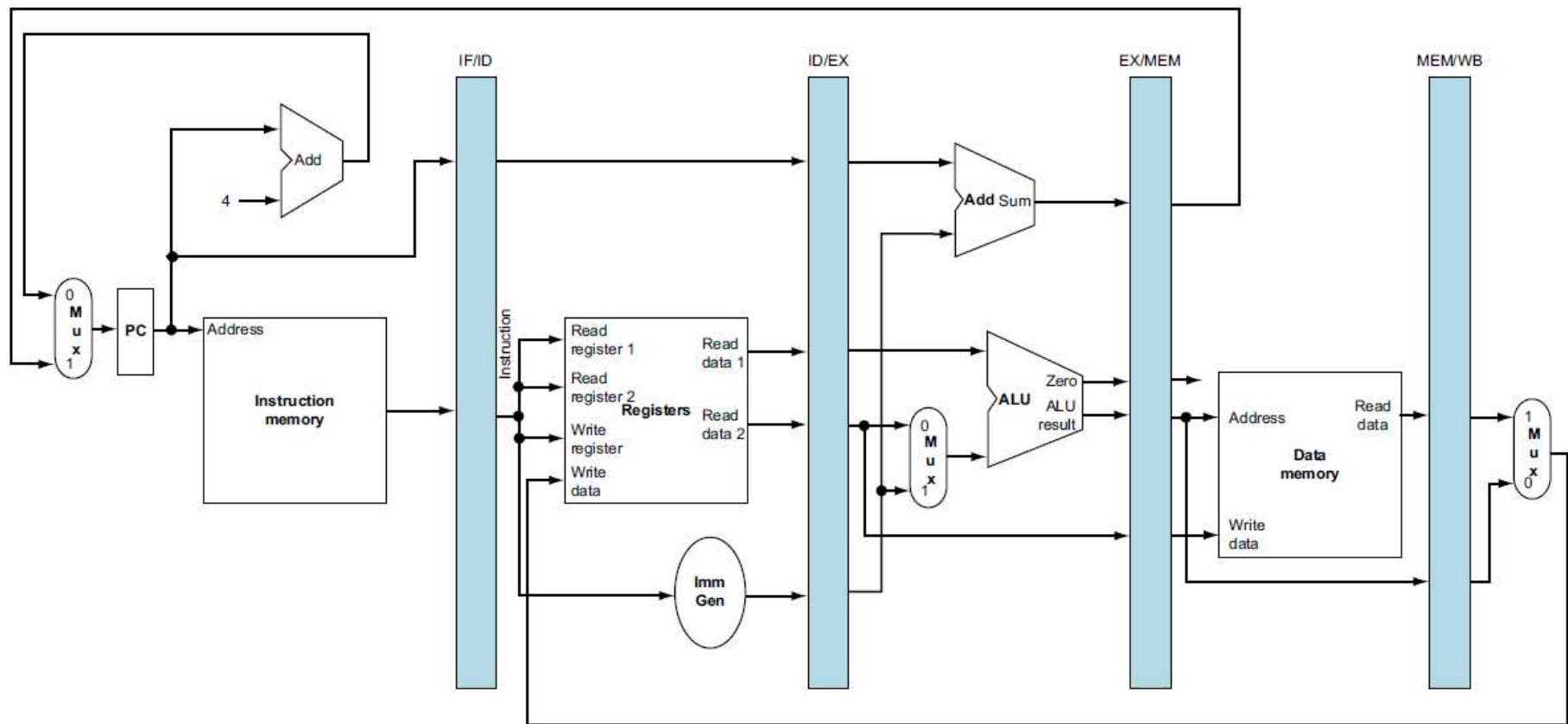
2. Proiectarea caii de date (data path) cu pipeline



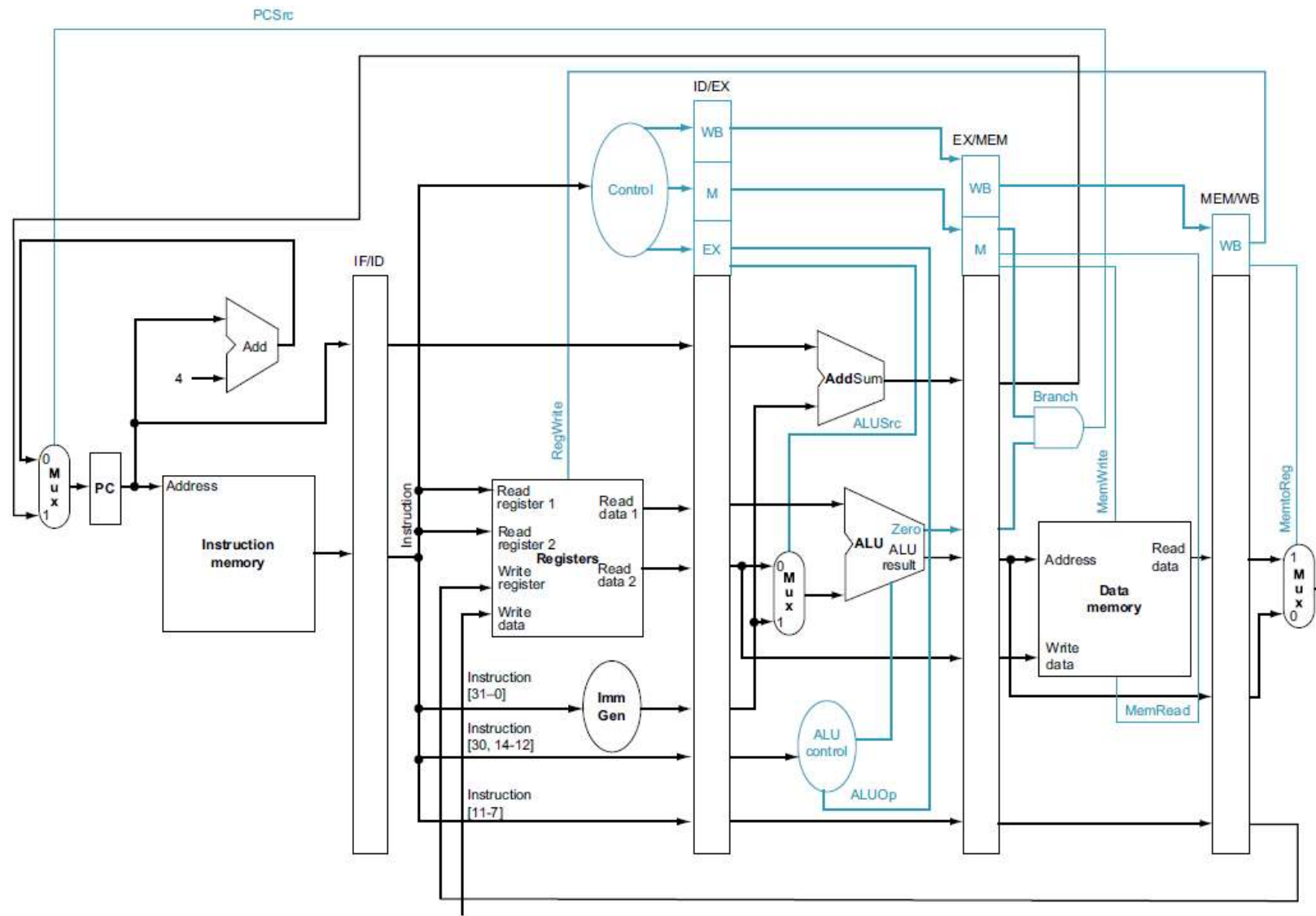
CLK $\square$ $\square$ $\square$ $\square$

E - extragere cod instructiune
D - decodare (extragere op. surse din set registri)
M - lucru cu memorie date si op. ALU
R - reintorcere, rezultat





Timp	t1	t2	t3	t4	t5	t6	t7	t8	
<i>Instr_1</i>	IF_1	ID_1	EX_1	MEM_1	WB_1				
<i>Instr_2</i>		IF_2	ID_2	EX_2	MEM_2	WB_2			
<i>Instr_3</i>			IF_3	ID_3	EX_3	MEM_3	WB_3		
<i>Instr_4</i>				IF_4	ID_4	EX_4	MEM_4	WB_4	



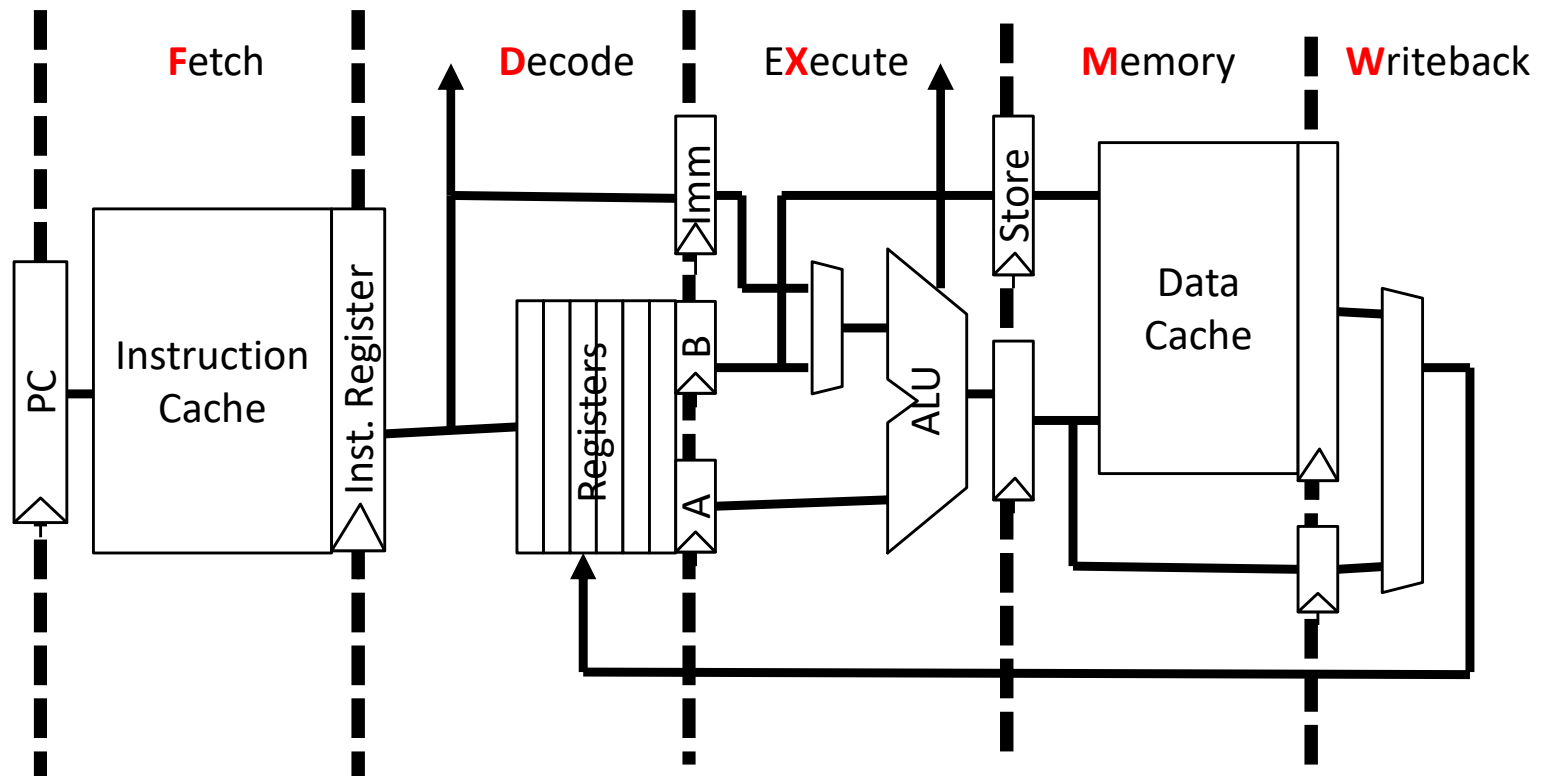
“Iron Law” of Processor Performance

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} * \frac{\text{Cycles}}{\text{Instruction}} * \frac{\text{Time}}{\text{Cycle}}$$

- Instructions per program depends on source code, compiler technology, and ISA
- Cycles per instructions (CPI) depends on ISA and μ architecture
- Time per cycle depends upon the μ architecture and base technology

Microarchitecture	CPI	cycle time
Microcoded	>1	short
Single-cycle unpipelined	1	long
Pipelined	1	short

Classic 5-Stage RISC Pipeline



*This version designed for regfiles/memories
with synchronous reads and writes.*

Instructions interact with each other in pipeline

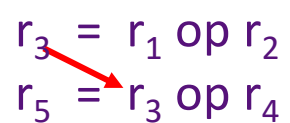
- An instruction in the pipeline may need a resource being used by another instruction in the pipeline → *structural hazard*
- An instruction may depend on something produced by an earlier instruction
 - Dependence may be for a data value
→ *data hazard*
 - Dependence may be for the next instruction's address
→ *control hazard (branches, exceptions)*
- Handling hazards generally introduces bubbles into pipeline and reduces ideal $CPI > 1$

Types of Data Hazards

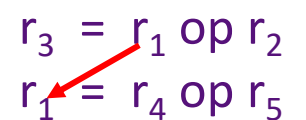
Consider executing a sequence of register-register instructions of type:

$$r_k = r_i \text{ op } r_j$$

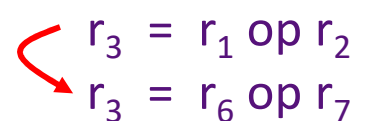
Data-dependence

$$\begin{array}{ll} r_3 = r_1 \text{ op } r_2 & \text{Read-after-Write} \\ r_5 = r_3 \text{ op } r_4 & \text{(RAW) hazard} \end{array}$$


Anti-dependence

$$\begin{array}{ll} r_3 = r_1 \text{ op } r_2 & \text{Write-after-Read} \\ r_1 = r_4 \text{ op } r_5 & \text{(WAR) hazard} \end{array}$$


Output-dependence

$$\begin{array}{ll} r_3 = r_1 \text{ op } r_2 & \text{Write-after-Write} \\ r_3 = r_6 \text{ op } r_7 & \text{(WAW) hazard} \end{array}$$


END 20 ian 2026